CS771 : Introduction to Machine Learning

Major Assignment-1

Aarsh Jain
230015
aarshjain23@iitk.ac.in

Aayush Kumar
230027
aayushk23@iitk.ac.in

Bikramjeet Singh
230298
bsingh23@iitk.ac.in

Hitarth Makawana
230479
hitarthkm23@iitk.ac.in

Kulshreshth Chikara
230586
kulshresh23@iitk.ac.in

Ronav Puri
230815
ronavgp23@iitk.ac.in

This report provides details and solutions for Major Assignment-1 of the course **CS771 (2025-26 I)**. The assignment consists of two distinct problems:

1. **Hiring using Semi-Parametric Regression**: This section focuses on modelling a hiring problem which is inherently non-linear using kernel polynomials and training it through **kernel ridge regression**.

2. **Delay Recovery by Inverting a XOR Arbiter PUF**: This section explores the use of the Kronecker Delta of two models, each belonging to one of the two PUFs in a XOR PUF, to generate a possible set of delays that produce the same input vector.

This report outlines the methods and approaches we followed for these two problems, supplemented by graphs and code examples where necessary.

1 Introduction

The first part of this assignment involves deriving and tuning a custom kernel function that effectively captures the semi-parametric structure of the given data. The necessary calculation involved alongwith the underlying logic has been provided.

The second part involves working with Arbiter-based PUFs. Given a high-dimensional linear model corresponding to the Kronecker Delta of the models of two 32-stage Arbiter PUFs (whose output whom is XORed to get the final output), the goal is to reconstruct the underlying 256 non-negative stage delays that generate the same model.

The report is organised as follows:

- Section 2 describes the derivation, implementation, and tuning of the polynomial kernel for semi-parametric regression problem.
- Section 3 details the mathematical derivation and the inversion algorithms for recovering Arbiter PUF delays.

2 Semi-Parametric Regression Using Polynomial Kernels

The problem aims at finding y, the number of staff members required to produce a video. It depends upon the following 3 factors:

1. $x \in \mathbb{R}^+$: Length of the video
2. $z_1 \in [0, 1]$: fraction of audience interested in the topic of the video
3. $z_2 \in [0, 1]$: difficulty level of the subject matter being discussed

It is known that:

$$y = w(\mathbf{z})\, x + b,$$

where $\mathbf{z} = [z_1, z_2]^\top \in [0,1]^2$ represents collectively the auxiliary features z_1, z_2 and b is the bias.

To model the dependence of y on $\mathbf{z}$, we note that $w(\mathbf{z})$ in the original formulation is a non-linear and potentially complex function of $\mathbf{z}$.

To address this, we introduce a *semi-parametric regression* approach, which assumes that $w(\mathbf{z})$ can be expressed as a linear model in a high-dimensional feature space:

$$w(\mathbf{z}) = \mathbf{p}^\top \phi(\mathbf{z}),$$

where $\phi(\mathbf{z})$ maps $\mathbf{z} \in \mathbb{R}^2$ into a higher-dimensional Hilbert space $\mathcal{H}$, and $\mathbf{p}$ is a corresponding parameter vector in that space. This transformation allows us to replace the difficult non-linear dependence in $\mathbf{z}$ with a linear dependence in $\phi(\mathbf{z})$, which can be handled efficiently using kernel methods. Thus, the overall model becomes:

$$y = \mathbf{p}^\top \phi(\mathbf{z})\, x + b,$$

A polynomial kernel,

$$K(\mathbf{z}_1, \mathbf{z}_2) = (\mathbf{z}_1^\top \mathbf{z}_2 + c)^d,$$

is chosen as it is mentioned in the problem statement that it is sufficient. The hyperparameters c (coefficient) and d (degree) are tuned empirically to optimise model performance.

From semi-parametric to pure kernel regression. Scikit-learn's `kernel_ridge` does not learn an explicit bias term, so we embed the bias into the feature map. Define the augmented map

$$\psi : \mathbb{R} \times \mathbb{R}^2 \longrightarrow \tilde{\mathcal{H}}, \qquad \psi(x, \mathbf{z}) \;=\; \begin{bmatrix} x\,\phi(\mathbf{z}) \\ 1 \end{bmatrix},$$

and augmented parameter vector $\tilde{\mathbf{p}} = \begin{bmatrix} \mathbf{p} \\ b \end{bmatrix}$. Then

$$\tilde{\mathbf{p}}^\top \psi(x, \mathbf{z}) = \mathbf{p}^\top \phi(\mathbf{z})\, x + b = y,$$

so the bias b is represented by the constant feature "1".

As kernel ridge depends only on inner products, the required kernel on pairs $((x_1, \mathbf{z}_1), (x_2, \mathbf{z}_2))$ is

$$\tilde{K}\big((x_1, \mathbf{z}_1), (x_2, \mathbf{z}_2)\big) = \big\langle \psi(x_1, \mathbf{z}_1),\, \psi(x_2, \mathbf{z}_2) \big\rangle = \big\langle x_1 \phi(\mathbf{z}_1),\, x_2 \phi(\mathbf{z}_2) \big\rangle + \langle 1, 1 \rangle.$$

We know that for constants a,b in an inner product we can write:

$$\langle a\,\phi(\mathbf{x}),\, b\,\phi(\mathbf{y}) \rangle = a\, b\, \langle \phi(\mathbf{x}),\, \phi(\mathbf{y}) \rangle.$$

Hence,

$$\langle x_1\, \phi(\mathbf{z_1}),\, x_2\, \phi(\mathbf{z_2}) \rangle = x_1\, x_2\, \langle \phi(\mathbf{z_1}),\, \phi(\mathbf{z_2}) \rangle.$$

Using $\langle \phi(\mathbf{z}_1), \phi(\mathbf{z}_2) \rangle = K(\mathbf{z}_1, \mathbf{z}_2)$ we obtain

$$\boxed{\tilde{K}\big((x_1, \mathbf{z}_1), (x_2, \mathbf{z}_2)\big) = x_1\, x_2\, K(\mathbf{z}_1, \mathbf{z}_2) \;+\; 1 = x_1\, x_2\, (\mathbf{z}_1^\top \mathbf{z}_2 + c)^d + 1.}$$

The key intuition is that we do not want to work with the explicit feature map $\phi(\mathbf{z})$ or with the constructed map $\psi(x, \mathbf{z})$, since these live in a high–dimensional Hilbert space and are never computed directly (essence of using Kernel methods). Instead, kernel methods allow us to replace inner products of the form $\langle \phi(\mathbf{z}_1), \phi(\mathbf{z}_2) \rangle$ with a known kernel function $K(\mathbf{z}_1, \mathbf{z}_2)$.

By designing $\tilde{K}$ so that

$$\tilde{K}\big((x_1, \mathbf{z}_1), (x_2, \mathbf{z}_2)\big) = \langle \psi(x_1, \mathbf{z}_1), \psi(x_2, \mathbf{z}_2) \rangle,$$

we ensure that kernel ridge regression implicitly performs the correct semi–parametric inner product in the high–dimensional space, without ever computing ψ itself. Thus, $\tilde{K}$ is constructed entirely from the known kernel K..

Kernel Implementation. The assignment requires the code to calculate a Gram Matrix which contains pairwise kernel values of two sets of data points. The following Python function computes the Gram matrix (The values of c and d here are just for representation not the most optimum values we found out):

```
def my_kernel(X1, Z1, X2, Z2):
    c, d = 1.0, 2
    Kz = (Z1 @ Z2.T + c) ** d
    Kx = X1 @ X2.T
    G  = Kx * Kz + 1.0
    return G
```

This essentially performs the required inner product calculation based on the new Kernel $\tilde{K}$ between pairs of data points - one from the first set and second from the other. The Gram Matrix generated here is used with `KernelRidge(kernel='precomputed')` to train the regression model.

Hyperparameter Tuning and Results. The kernel parameters (c, d) were varied systematically. For each combination, we computed:

- Kernel computation time (t_{kernel}).
- Model training time (t_{train}).
- Test R^2 score averaged over five random trials.

The observed trends from the heatmap can be summarised directly as follows:

- For most combinations of (c, d), the R^2 score remains extremely high and lies in a very narrow band around 0.9697.
- When d is small (around 2 or 3), the performance is consistently strong for all values of c.
- As we move to larger values of d (especially $d = 4$ and $d = 5$), the scores in the first row ($c = 0.1$) begin to drop. Also we got numerical warnings if we increased d beyond $d = 3$ due to singular Gram Matrices.
- For moderate and large c values (the remaining rows), the performance stays almost unchanged across all degrees, with the heatmap showing nearly identical values across each row.

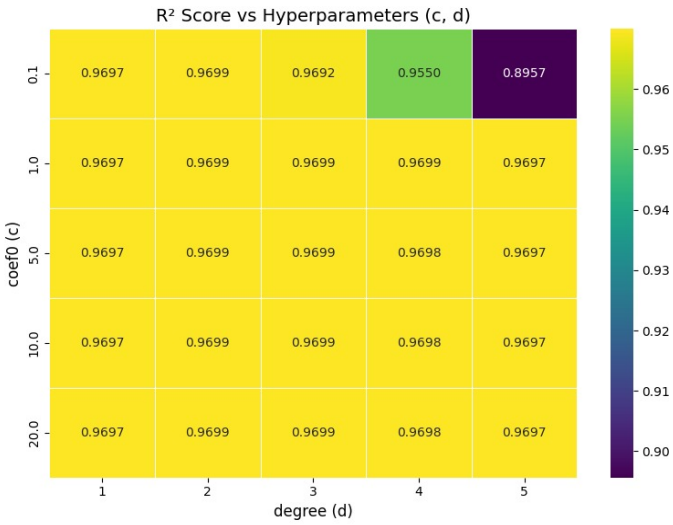

Figure 1: R^2 scores across polynomial kernel hyperparameters (c, d). The heatmap shows that almost all (c, d) pairs yield very similar performance, with the only visible dip occurring in the top row ($c = 0.1$) for higher degrees $d = 4$ and $d = 5$. Apart from this isolated region, the scores remain uniformly high across the matrix.

As shown in Figure 1, the highest R^2 values occur near $d = 2$ and $d = 3$. To further analyze the influence of c for these degrees, we plotted R^2 and training time against c below.

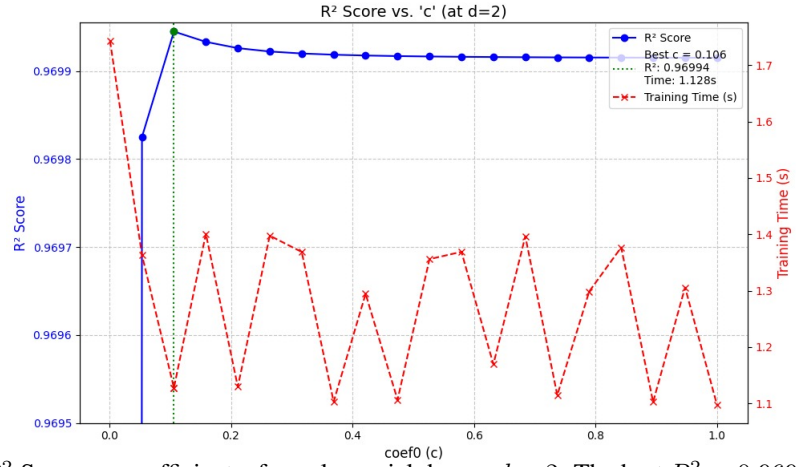

Figure 2: R^2 Score vs. coefficient c for polynomial degree $d = 2$. The best $R^2 \approx 0.9699$ is achieved at $c = 0.106$ with a training time of approximately 1.13 s. This configuration provides high predictive accuracy while maintaining relatively low computational cost.

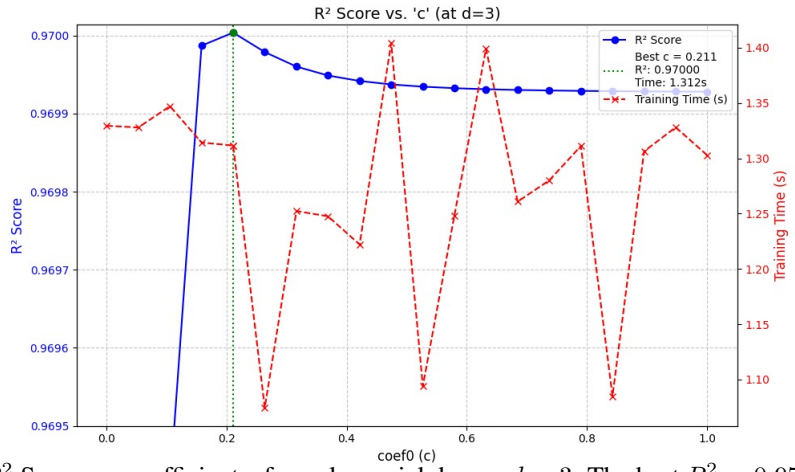

Figure 3: R^2 Score vs. coefficient c for polynomial degree $d = 3$. The best $R^2 \approx 0.9700$ occurs at $c = 0.211$, with a higher training time of about 1.31 s. While this setup gives a marginally better R^2, it demands more computation time compared to $d = 2$.

In summary, both $d = 2$ and $d = 3$ yield near-identical predictive performance. However, $d = 2$ achieves comparable accuracy with shorter training time, making it the more efficient and stable choice for the final kernel configuration.

Conclusion. Overall, the best-performing setting was obtained at $c = 0.1$ and $d = 2$, which achieved the highest R^2 score of approximately 0.9699 with an average training time of around 1.13 seconds.

3 Inversion of a XOR-Arbiter PUF

An Arbiter PUF consists of multiple stages, each with four delays (p_i, q_i, r_i, s_i). For each stage, we define (from class notes):

$$\alpha_i = \frac{p_i - q_i + r_i - s_i}{2}, \quad \beta_i = \frac{p_i - q_i - r_i + s_i}{2}.$$

These quantities form a $(k+1)$-dimensional linear model $\mathbf{u}$ (in our case $k = 32$):

$$u_0 = \alpha_0, \quad u_i = \alpha_i + \beta_{i-1} \text{ for } 1 \le i \le k-1, \quad u_k = \beta_{k-1}.$$

For a XOR of two such PUFs (each with 32 stages), the combined 1089-dimensional linear model is given by

$$\mathbf{w} = \mathbf{u} \otimes \mathbf{v},$$

where $\otimes$ denotes the Kronecker delta.

Detailed Derivation and Inversion Procedure. We start with two Arbiter PUFs, each having $k = 32$ stages. For a single PUF, each stage has four internal delays (a_i, b_i, c_i, d_i) which determine how the signals propagate through the multiplexers forming the PUFs. The linear model corresponding to a single PUF is represented by a $(k+1)$-dimensional vector $\mathbf{u} = [u_0, u_1, \ldots, u_k]^\top$, constructed recursively from the intermediate quantities α_i and β_i:

$$u_0 = \alpha_0, \qquad u_i = \alpha_i + \beta_{i-1} \; (1 \le i \le k-1), \qquad u_k = \beta_{k-1}.$$

Here,

$$\alpha_i = \frac{a_i - b_i + c_i - d_i}{2}, \quad \beta_i = \frac{a_i - b_i - c_i + d_i}{2}.$$

Why the model has 33 dimensions. From the lecture notes on Arbiter PUF linear modeling, the delay difference after the final (k-th) stage is given as:

$$\Delta_k = w_0 x_0 + w_1 x_1 + \cdots + w_{k-1} x_{k-1} + \beta_k = \mathbf{w}^\top \mathbf{x} + b.$$

Here,

$$x_i = d_i \cdot d_{i+1} \cdots d_k, \quad w_0 = \alpha_0, \quad w_i = \alpha_i + \beta_{i-1} \text{ for } i > 0,$$

and the last term β_k is not multiplied by any x_i. It therefore acts as an additive constant — the bias term b in the equivalent linear model.

Thus, although the PUF has $k = 32$ stages, the effective linear representation contains $(k+1)$ parameters: k weights corresponding to stage contributions, and one additive bias term. Hence, the model vector $\mathbf{u}$ has 33 components.

When two such 33-dimensional models (each corresponding to one PUF in the XOR construction) are combined through the Kronecker product,

$$\mathbf{w} = \mathbf{u} \otimes \mathbf{v},$$

the resulting XOR-PUF model has $33 \times 33 = 1089$ coefficients.

Intuition behind our approach. Given $\mathbf{w}$, we must obtain a valid pair of vectors $(\mathbf{u}, \mathbf{v})$ and then reconstruct the underlying stage delays. However, this inverse problem is not unique: if we scale one vector and inversely scale the other, $\frac{1}{c}\mathbf{u} \otimes (c\mathbf{v})$ gives the same $\mathbf{w}$. Since the delays are not constrained to any particular range, we can exploit this freedom and *fix one reference coordinate* in any of the two vectors to break the ambiguity. Since we have been given the Kronecker Delta $\mathbf{w}$ we can use this fixed coordinate as a way to find the other unknowns as described below.

Explicit construction. Let the two single–PUF model vectors be written explicitly as

$$\mathbf{u} = \begin{bmatrix} u_0 \\ u_1 \\ \vdots \\ u_{32} \end{bmatrix} \in \mathbb{R}^{33}, \qquad \mathbf{v} = \begin{bmatrix} v_0 \\ v_1 \\ \vdots \\ v_{32} \end{bmatrix} \in \mathbb{R}^{33}.$$

Their Kronecker combination is the 33×33 table of pairwise products

$$W \;=\; \big[\, W_{ij} \,\big]_{i,j=0}^{32}, \qquad W_{ij} \;=\; u_i \, v_j,$$

which is just a reshaped view of the given 1089-dimensional vector $\mathbf{w}$.

Our hand-crafted solver fixes the very first coordinate of $\mathbf{u}$,

$$\boxed{u_0 = 1}.$$

This immediately gives the entire vector $\mathbf{v}$ from the first column of W:

$$v_i \;=\; W_{i0} \quad \text{for } i = 0, 1, \ldots, 32,$$

because $W_{i0} = v_i u_0 = v_i$ when $u_0 = 1$.

With $\mathbf{v}$ known, pick any index $i^\star$ such that $v_{i^\star} \neq 0$ (in practice we use $i^\star = 0$ unless $v_0 = 0$). Then the whole vector $\mathbf{u}$ is read off from the $i^\star$-th row:

$$u_j \;=\; \frac{W_{i^\star j}}{v_{i^\star}} \quad \text{for } j = 0, 1, \ldots, 32,$$

since $W_{i^\star j} = v_{i^\star} u_j$.

This procedure is consistent with the scalar scaling freedom: for any $c \neq 0$, the pair $\left(\frac{1}{c}\mathbf{u},\, c\mathbf{v}\right)$ produces the same table W. By fixing $u_0 = 1$ we remove that ambiguity and obtain a deterministic pair $(\mathbf{u}, \mathbf{v})$ that exactly reproduces the given $\mathbf{w}$.

Recovering α_i and β_i from the recovered vectors $\mathbf{u}$ and $\mathbf{v}$. The assignment does not require the recovered delays to be unique, we are free to choose any (α_i, β_i) that leads to a valid set of stage delays.

Following this idea, we use a simple and consistent choice:

$$\alpha_i = u_i \quad \text{for all } 0 \leq i \leq 31,$$

and set all β_i to zero except the last one:

$$\beta_{31} = u_{32}.$$

This construction provides a convenient assignment that always yields a valid set of delays and exactly reproduces the given $\mathbf{w}$ when the forward model is applied again. The same assignment is made independently for the vector $\mathbf{v}$.

Constructing non-negative stage delays. Each stage of a PUF is determined only by the delay *differences*

$$(a_i - b_i) = \alpha_i + \beta_i, \qquad (c_i - d_i) = \alpha_i - \beta_i.$$

Since these differences may be positive or negative, but the physical delays must be non-negative, we simply split each difference into positive and negative parts:

$$a_i = \max(0,\, a_i - b_i), \qquad b_i = \max(0,\, -(a_i - b_i)),$$
$$c_i = \max(0,\, c_i - d_i), \qquad d_i = \max(0,\, -(c_i - d_i)).$$

This guarantees a valid non-negative assignment for all four delays at every stage. The same construction is used for the second PUF, giving one consistent set of 256 delays whose forward model exactly reproduces the given vector $\mathbf{w}$.

Validation. Substituting these delays back into the forward relations exactly reproduces the same $\mathbf{w} = \mathbf{u} \otimes \mathbf{v}$. This confirms the correctness of the reconstruction.

Results. Using the proposed decoding approach, the inversion step ran extremely quickly, finishing in about 1.16×10^{-4} seconds on the evaluation input. The difference between the reconstructed model and the provided one was on the order of 10^{-16}, which is essentially zero apart from unavoidable floating-point rounding effects. This shows that the recovered delays reproduce the original model almost exactly.

Conclusion.

- The method is completely deterministic once a reference coordinate (here $u_{32} = 1$) is fixed.
- The resulting solution provides one of infinitely many valid realizations of the 256 underlying delays.
- Hence, using a simple hand-crafted solver that exploits the absence of delay constraints, we obtain a closed-form constructive inversion of the XOR-Arbiter PUF model.